

Satpuda Core New-User Accounting Logic Implementation Guide

Clean rebuild blueprint for Sales, Sales History, Purchase, Purchase History, Account Clear, GST Filing, Profit Calculation, Discounts, and future Sales/Purchase Returns. Existing old records are intentionally not considered.

Generated: April 25, 2026. Scope: documentation/guide only. No application source code changes are included in this PDF.

1. Main Rule for the Rebuild

For new users, rebuild the logic as one shared accounting engine. Sales, Purchase, Sales History, Purchase History, GST Reports, and Returns should all read/write through the same formulas and ledger rules. Do not keep separate calculations inside each page.

- Use Decimal for all money values. Avoid float for money.
- Store all historical tax, discount, cost, and rate snapshots at transaction time.
- Keep invoice facts immutable where possible. Edits should void/repost or create correction entries instead of silently mutating old accounting history.
- Use ledgers for account clear and due/credit. Do not update old rows just to show a green cleared status.
- Use stock ledger entries for stock movement. Do not rely only on medicines.stock_qty updates.

2. Shared Money and Rounding Rules

2.1 Decimal Rules

```
money(x) = Decimal(x).quantize(Decimal("0.01"), ROUND_HALF_UP)
qty(x) = Decimal(x)
rate(x) = Decimal(x)
```

- Round line taxable value and GST to 2 decimals using ROUND_HALF_UP.
- For invoice totals, sum rounded line values, then apply invoice round-off.
- If line rounding creates a one/two paise mismatch, adjust the largest taxable line and store the adjustment as tax_rounding_adjustment.

2.2 Invoice Round-Off

```
pre_round_total = sum(line_total_after_tax) + other_charges - header_discount_after_tax
rounded_total = Decimal(pre_round_total).quantize(Decimal("1"), ROUND_HALF_UP)
round_off = rounded_total - pre_round_total
grand_total = pre_round_total + round_off
```

- Store round_off separately on the invoice header.
- Do not hide round-off inside GST taxable value unless your tax advisor explicitly requires it.

3. GST Formula Library

3.1 GST-Exclusive Rate Formula

```
gross_before_discount = qty * rate_ex_gst
item_discount_amount = discount_from_input(gross_before_discount)
taxable_before_bill_discount = gross_before_discount - item_discount_amount
allocated_bill_discount = bill_discount * taxable_before_bill_discount / total_taxable_before_bill_discount
taxable_value = taxable_before_bill_discount - allocated_bill_discount
gst_amount = taxable_value * gst_rate / 100
line_total = taxable_value + gst_amount
```

3.2 GST-Inclusive MRP Formula

```
gross_inclusive = qty * rate_including_gst
item_discount_amount_inclusive = discount_from_input(gross_inclusive)
net_inclusive_before_bill_discount = gross_inclusive - item_discount_amount_inclusive
allocated_bill_discount_inclusive = bill_discount * net_inclusive_before_bill_discount /
total_net_inclusive_before_bill_discount
```

```

net_inclusive = net_inclusive_before_bill_discount - allocated_bill_discount_inclusive
taxable_value = net_inclusive * 100 / (100 + gst_rate)
gst_amount = net_inclusive - taxable_value
line_total = net_inclusive

```

3.3 CGST/SGST/IGST Split

```

if supply_type == "intra_state":
    cgst_amount = gst_amount / 2
    sgst_amount = gst_amount / 2
    igst_amount = 0
else:
    cgst_amount = 0
    sgst_amount = 0
    igst_amount = gst_amount

```

- For a local medical store, default can be intra_state. Keep supply_type on invoice for future expansion.

4. Discount Rules

Discount type	Best storage	Formula/use
Item percent discount	discount_type=item_percent, discount_input=%, discount_amount	discount_amount = base * percent / 100
Item rupee discount	discount_type=item_amount, discount_input=amount, discount_amount	discount_amount = entered amount
Bill percent discount	bill_discount_type=percent, bill_discount_input=%, bill_discount_amount	bill_discount_amount = eligible_base_total * percent / 100
Bill rupee discount	bill_discount_type=amount, bill_discount_input=amount, bill_discount_amount	bill_discount_amount = entered amount

- For GST filing, prefer discount before GST and allocate bill discount line-wise.
- Store both input and calculated rupee amount so edit screens do not confuse amount with percentage.
- Each invoice item should store item_discount_amount and bill_discount_allocated.

5. New Database Design for New Users

You may keep existing table names if you prefer, but the important point is to add the missing accounting facts and ledgers from day one.

Table	Purpose / required fields
customers	id, name, phone, address, gstin optional, opening_balance, is_active
suppliers	id, name, phone, address, gstin, dl_numbers, opening_balance, is_active
sales	id, bill_no, customer_id, bill_date, supply_type, taxable_total, cgst_total, sgst_total, igst_total, gst_total, item_discount_total, bill_discount_type/input/amount, round_off, grand_total, invoice_status
sales_items	sale_id, medicine_id, batch_no, qty, unit_factor, rate_including_gst, gross_inclusive, discount fields, taxable_value, gst_rate, GST amounts, line_total, cost_rate_ex_gst_snapshot, hsn_code
purchases	id, purchase_no, supplier_id, bill_number, purchase_date, rate_tax_mode(exclusive/inclusive), taxable_total, input_cgst_total, input_sgst_total, input_igst_total, input_gst_total, discounts, round_off, grand_total, invoice_status
purchase_items	purchase_id, medicine_id, qty, free_qty, rate_ex_gst or inclusive, discount fields, taxable_value, gst_rate, input GST amounts, line_total, mrp, batch, expiry, hsn_code
payments	id, party_type, party_id, invoice_type nullable, invoice_id nullable, method, amount, paid_at, reference_no
party_ledger	id, party_type, party_id, source_type, source_id, signed_amount, balance_after, entry_date, note
invoice_allocations	payment/credit/return allocation to invoice. Fields: invoice_type, invoice_id, source_type, source_id, amount
stock_ledger	medicine_id, batch_no, source_type, source_id, qty_in, qty_out, cost_rate_ex_gst, created_at
sale_returns / sale_return_items	linked to original sale/sales_items with returned tax and stock values
purchase_returns / purchase_return_items	linked to original purchase/purchase_items with reversed input GST and stock values

6. Account Clear Logic

6.1 Universal Signed Balance

Use a signed party balance. This keeps Sales and Supplier logic consistent and makes history clear without rewriting old invoices.

Party type	Positive balance means	Negative balance means	Zero means
Customer	Customer owes the store	Customer has advance/credit	Customer account clear
Supplier	Store owes supplier	Store has supplier advance/debit credit	Supplier account clear

6.2 Customer Ledger Entries

```
On sale invoice save:
ledger +grand_total # customer owes store
On customer payment:
ledger -payment_amount
On sales return credit note:
ledger -return_total
On cash refund to customer:
ledger +refund_amount # refund consumes customer credit
customer_balance = sum(customer ledger signed_amount)
if customer_balance > 0: account_status = "DUE"
elif customer_balance < 0: account_status = "CREDIT"
else: account_status = "CLEAR"
```

6.3 Supplier Ledger Entries

```
On purchase invoice save:
ledger +grand_total # store owes supplier
On supplier payment:
ledger -payment_amount
On purchase return / debit note:
ledger -return_total
On supplier refund to store:
ledger +refund_amount # refund reduces debit/advance effect if needed
supplier_balance = sum(supplier ledger signed_amount)
if supplier_balance > 0: account_status = "PAYABLE"
elif supplier_balance < 0: account_status = "ADVANCE_OR_CREDIT"
else: account_status = "CLEAR"
```

6.4 Invoice Clear vs Account Clear

- invoice_status is per invoice: OPEN, PARTIAL, CLEARED, RETURNED, PARTIAL_RETURN, VOID.
- account_status is per party from ledger balance: DUE/PAYABLE, CREDIT/ADVANCE, CLEAR.
- History pages should show both: invoice_due and party_balance_after_invoice.
- Do not mark all old rows account_cleared = 0 when a new due invoice is created. Old invoices remain cleared; the party account becomes due because of the new ledger entry.

6.5 Payment Allocation Formula

```
remaining_payment = payment_amount
for invoice in oldest_open_invoices:
allocation = min(invoice.remaining_due, remaining_payment)
create invoice_allocations row
invoice.remaining_due -= allocation
remaining_payment -= allocation
if invoice.remaining_due == 0: invoice_status = "CLEARED"
if remaining_payment > 0:
keep as party credit/advance in ledger
```

- Use FIFO allocation by default. Add manual allocation later if needed.

7. Sales Page: Correct Flow and Formulas

7.1 Load Customer

- Load customer current balance from party_ledger, not from latest sales row.

- If balance > 0 show Previous Due. If balance < 0 show Available Credit. If zero show Clear.
- Available customer credit can be auto-applied to the sale or applied through an explicit Credit Used field.

7.2 Add Sale Item

```
sale_rate = MRP / unit_factor for tablet/bolus, otherwise MRP
gross_inclusive = qty * sale_rate
item_discount_amount = gross_inclusive * item_discount_percent / 100
net_inclusive_before_bill_discount = gross_inclusive - item_discount_amount
```

- Store medicine GST percent copied at sale time.
- Store cost_rate_ex_gst_snapshot from batch/purchase cost for profit.
- Store item discount percent and amount.

7.3 Sale Total

```
bill_discount_amount = calculate from bill discount input
allocate bill_discount_amount across item net inclusive amounts
for each item:
net_inclusive = net_inclusive_before_bill_discount - allocated_bill_discount
taxable_value = net_inclusive * 100 / (100 + gst_rate)
gst_amount = net_inclusive - taxable_value
line_total = net_inclusive
subtotal_inclusive = sum(line_total)
round_off = nearest_rupee(subtotal_inclusive) - subtotal_inclusive
grand_total = subtotal_inclusive + round_off
```

7.4 Sale Payment and Save

```
customer_old_balance = ledger_balance(customer)
available_credit = max(0, -customer_old_balance)
credit_used = min(available_credit, grand_total)
amount_due_before_new_payment = grand_total - credit_used
paid_now = cash_paid + online_paid
invoice_due = max(0, amount_due_before_new_payment - paid_now)
new_customer_balance = customer_old_balance + grand_total - credit_used - paid_now
```

- Save sale header and sale items.
- Create stock_ledger qty_out for each item.
- Create party_ledger +grand_total for sale invoice.
- Create party_ledger -credit_used if old customer credit is explicitly allocated to this invoice.
- Create payment rows and party_ledger negative entries for cash/online paid.
- Update invoice_status from allocations: CLEARED if invoice_due is zero, PARTIAL if paid/credit used, OPEN if no payment.

8. Sales History Page: Correct Flow

- Read stored sale header and item snapshots. Never recalculate GST from current medicines table.
- Show columns: Bill No, Date, Customer, Taxable Total, GST Total, Round Off, Grand Total, Paid Allocated, Credit Used, Return Value, Invoice Due, Customer Balance After, Invoice Status.
- Due Only filter should use latest customer ledger balance > 0 or invoice.remaining_due > 0, depending on selected report mode.
- Credit Only filter should use latest customer ledger balance < 0.
- Paid/Cleared filter should use invoice_status = CLEARED.
- Editing should void/repost or create correction entries. It should not change old rows without rebuilding ledger, stock, GST, and profit effects.
- Deleting should be replaced with Void Bill. Void creates reversing ledger and stock entries and marks invoice_status = VOID.

8.1 Sales Profit Formula

```
sale_profit_line = sale_taxable_value - (qty_sold * cost_rate_ex_gst_snapshot)
sale_profit_invoice = sum(sale_profit_line)
return_profit_reversal = returned_taxable_value - (returned_qty * original_cost_rate_ex_gst_snapshot)
net_profit = sale_profit_invoice - return_profit_reversal
```

- Profit should exclude GST because GST collected is tax liability, not business profit.

9. Purchase Page: Correct Flow and Formulas

9.1 Load Supplier

- Load supplier balance from supplier party_ledger.
- If balance > 0 show payable due. If balance < 0 show supplier credit/advance. If zero show clear.

9.2 Add Purchase Item

Default purchase rate mode should be GST-exclusive, because supplier bills usually show taxable rate plus GST. Keep a rate_tax_mode field in case a supplier enters inclusive prices.

```
If GST-exclusive:
gross_taxable = qty * purchase_rate_ex_gst
item_discount_amount = discount_from_input(gross_taxable)
taxable_before_bill_discount = gross_taxable - item_discount_amount

If GST-inclusive:
gross_inclusive = qty * purchase_rate_including_gst
taxable_before_discount = gross_inclusive * 100 / (100 + gst_rate)
item_discount_amount = discount_from_input(taxable_before_discount)
taxable_before_bill_discount = taxable_before_discount - item_discount_amount
```

9.3 Purchase Total

```
bill_discount_amount = calculate from overall discount input
allocate bill_discount_amount across taxable_before_bill_discount
for each item:
taxable_value = taxable_before_bill_discount - allocated_bill_discount
input_gst_amount = taxable_value * gst_rate / 100
line_total = taxable_value + input_gst_amount
taxable_total = sum(taxable_value)
input_gst_total = sum(input_gst_amount)
round_off = nearest_rupee(sum(line_total)) - sum(line_total)
grand_total = sum(line_total) + round_off
```

- Store CGST/SGST/IGST per purchase item, not only GST percent.
- Free quantity increases stock but should have zero taxable amount unless supplier bill has a value for it.

9.4 Purchase Payment and Save

```
supplier_old_balance = ledger_balance(supplier)
available_supplier_credit = max(0, -supplier_old_balance)
credit_used = min(available_supplier_credit, grand_total)
amount_payable_before_payment = grand_total - credit_used
paid_now = cash_paid + online_paid
invoice_due = max(0, amount_payable_before_payment - paid_now)
new_supplier_balance = supplier_old_balance + grand_total - credit_used - paid_now
```

- Create purchase header and purchase item snapshots.
- Create stock_ledger qty_in for paid and free quantities.
- Create party_ledger +grand_total for supplier payable.
- Create payment rows and party_ledger negative entries for supplier payments.
- Allocate payments FIFO or directly to this invoice.

10. Purchase History Page: Correct Flow

- Read purchase header/item snapshots. Do not recalculate purchase totals from current medicine records.
- Show columns: Bill No, Date, Supplier, Taxable Total, Input GST, Round Off, Grand Total, Paid Allocated, Debit/Credit Used, Return Value, Invoice Due, Supplier Balance After, Invoice Status.
- Due Only should mean supplier current balance > 0 or invoice remaining due > 0, depending on filter mode.
- Supplier Due Report should group by supplier and show latest ledger balance only. Do not list multiple old running-balance rows as separate dues.
- Purchase edit should void/repost or post correction entries and should adjust stock by ledger reversal plus new stock entry.
- Purchase delete should become Void Purchase with reversing stock and supplier ledger entries.

11. GST Report Generation

11.1 Sales GST Report

Report column	Formula/source
Invoice No/Date	sales.bill_no, sales.bill_date
Customer/GSTIN	customers fields if B2B needed
HSN	sales_items.hsn_code
GST Rate	sales_items.gst_rate
Taxable Value	sum(sales_items.taxable_value - returned_taxable_value)
Output CGST/SGST/IGST	sum item GST fields less return GST fields
Invoice Value	sales.grand_total - sale_return_total

11.2 Purchase GST Report

Report column	Formula/source
Supplier Invoice No/Date	purchases.bill_number, purchases.purchase_date
Supplier/GSTIN	suppliers.gstin
HSN	purchase_items.hsn_code
GST Rate	purchase_items.gst_rate
Taxable Value	sum(purchase_items.taxable_value - purchase_return_taxable_value)
Input CGST/SGST/IGST	sum purchase item GST fields less purchase return GST fields
Invoice Value	purchases.grand_total - purchase_return_total

11.3 Net GST Summary

```
output_cgst = sales_cgst - sales_return_cgst
output_sgst = sales_sgst - sales_return_sgst
input_cgst = purchase_cgst - purchase_return_cgst
input_sgst = purchase_sgst - purchase_return_sgst
net_cgst_payable = output_cgst - input_cgst
net_sgst_payable = output_sgst - input_sgst
net_igst_payable = output_igst - input_igst
```

- Also provide rate-wise and HSN-wise summaries for audit.

12. Sales Return Compatibility

12.1 Sales Return Item Formula

```
return_qty <= original_qty - already_returned_qty
ratio = return_qty / original_qty
return_taxable_value = original_taxable_value * ratio
return_gst_amount = original_gst_amount * ratio
return_line_total = original_line_total * ratio
return_cost_value = original_cost_rate_ex_gst_snapshot * return_qty
```

- Create sale_return and sale_return_items rows linked to original sale item.
- Create stock_ledger qty_in for returned stock.
- Create party_ledger negative entry for return credit note.
- If cash is refunded, create refund payment and ledger entry that consumes the credit.
- Update original invoice status to PARTIAL_RETURN or RETURNED based on total returned quantity/value.

13. Purchase Return Compatibility

13.1 Purchase Return Item Formula

```
return_qty <= purchased_qty + free_qty - already_returned_qty - unavailable_qty_policy
ratio = return_qty / original_purchased_qty
return_taxable_value = original_taxable_value * ratio
```

```

return_input_gst = original_input_gst * ratio
return_line_total = original_line_total * ratio

```

- Create purchase_return and purchase_return_items linked to original purchase item.
- Create stock_ledger qty_out for returned stock.
- Create supplier party_ledger negative entry because payable reduces.
- If supplier gives cash refund, record payment/refund according to balance treatment.
- Reverse input GST in GST purchase return/debit note report.

14. Page-by-Page Calculation Ownership

Page	Should calculate locally?	Correct source of truth
Sales	Only preview. Save uses shared calculation service.	SaleCalculationResult + ledger posting transaction
Sales History	No tax/profit recalculation from current medicine data.	Saved invoice/item snapshots + ledger + stock ledger
Purchase	Only preview. Save uses shared calculation service.	PurchaseCalculationResult + ledger posting transaction
Purchase History	No recalculation except display aggregates from saved facts.	Saved purchase/item snapshots + ledger + stock ledger
GST Reports	Aggregate only from item tax facts and return facts.	sales_items, purchase_items, return_items
Profit Reports	Aggregate sale taxable value minus cost snapshots, adjusted by returns.	sales_items cost snapshot + return_items

15. Suggested Shared Services

- money.py: Decimal parsing, rounding, nearest_rupee, safe currency formatting.
- tax_calculator.py: calculate_line_tax, allocate_bill_discount, split_gst, invoice totals.
- sales_service.py: validate sale, save sale, post sale ledger, post sale stock movements.
- purchase_service.py: validate purchase, save purchase, post supplier ledger, post stock movements.
- ledger_service.py: get_party_balance, post_entry, allocate_payment_fifo, invoice_remaining_due.
- return_service.py: create sale return, create purchase return, reverse GST/profit/stock/account effects.
- report_service.py: GST sales report, GST purchase report, net GST summary, profit report, due report.

16. UI Rules to Keep History Always Clear

- Every save must be one database transaction: invoice rows, item rows, ledger rows, stock rows, payment rows, allocations.
- If any part fails, rollback the entire transaction.
- History pages should never write account-cleared flags just for display.
- Use invoice_status and ledger balance for colors: green clear, yellow invoice clear but party due from other invoices, red invoice/party due, blue/green credit.
- Avoid hard delete for posted invoices. Use VOID and reversal entries.
- When editing, show a correction workflow: Void old invoice and repost corrected invoice with a link to the original.

17. Required Test Cases Before Coding Returns

Test	Expected
Sale paid exact	Invoice CLEARED, customer balance unchanged after payment.
Sale partial paid	Invoice PARTIAL/OPEN, customer balance positive.

Test	Expected
Sale overpaid	Invoice CLEARED, customer balance negative credit. Next sale can use credit.
Purchase partial paid	Purchase PARTIAL/OPEN, supplier balance positive payable.
Supplier advance	Supplier balance negative. Next purchase uses advance correctly.
Mixed GST sale	Line-wise 0/5/12/18 percent GST totals match report.
Overall discount before GST	Discount allocated line-wise and GST reduced correctly.
Sales return partial	Stock increases, output GST reverses, customer balance reduces, profit reverses.
Purchase return partial	Stock decreases, input GST reverses, supplier payable reduces.
Void sale	Ledger and stock reverse; GST/profit reports exclude or show reversal clearly.
History after new due invoice	Old cleared invoices remain cleared; party account shows current due.

18. Final Build Recommendation

For a new-user rebuild, the best path is not to repair the existing per-page calculations one by one. Build the shared calculation and ledger layer first, then connect Sales and Purchase entry pages to it, then rebuild both history pages as readers of saved snapshots and ledgers. Once that foundation is stable, Sales Return, Purchase Return, GST reports, and profit reports become natural extensions instead of another set of fragile calculations.